Twitch Analytics
API 2

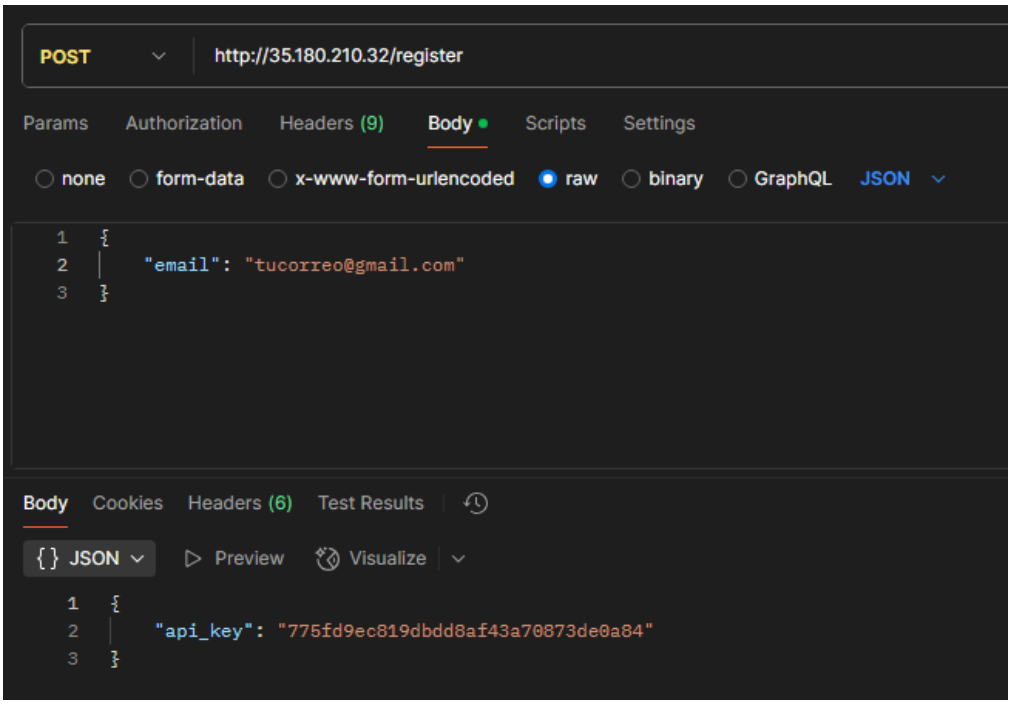
Índice

Introducción **3**
Explicación del código **5**
Estructura de la base de datos **6**
Problemas y dificultades encontradas **6**
Problemas en el código 6
Problemas en el servidor 7
Problemas en la BBDD 7
Implementación y ubicación del proyecto **7**

Introducción

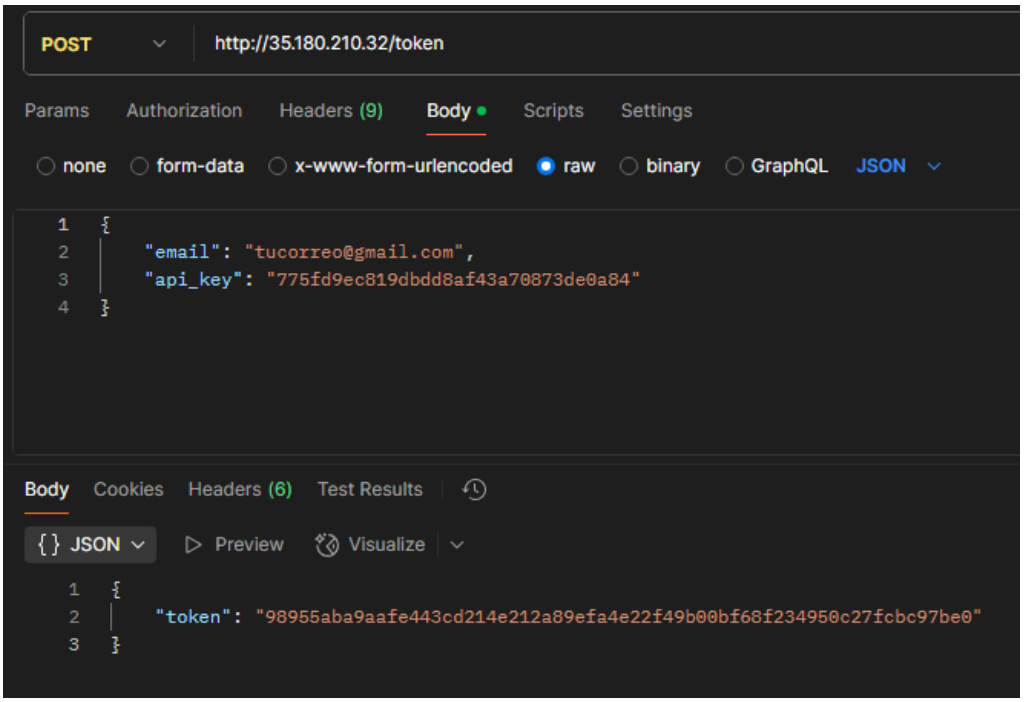
Este proyecto consiste en consultar información de la API de twitch a través de la API que hemos creado nosotros. Se pueden realizar las siguientes peticiones:

- **POST /register**: Registrarse en nuestra API. Para ello, debemos introducir un correo en el body:



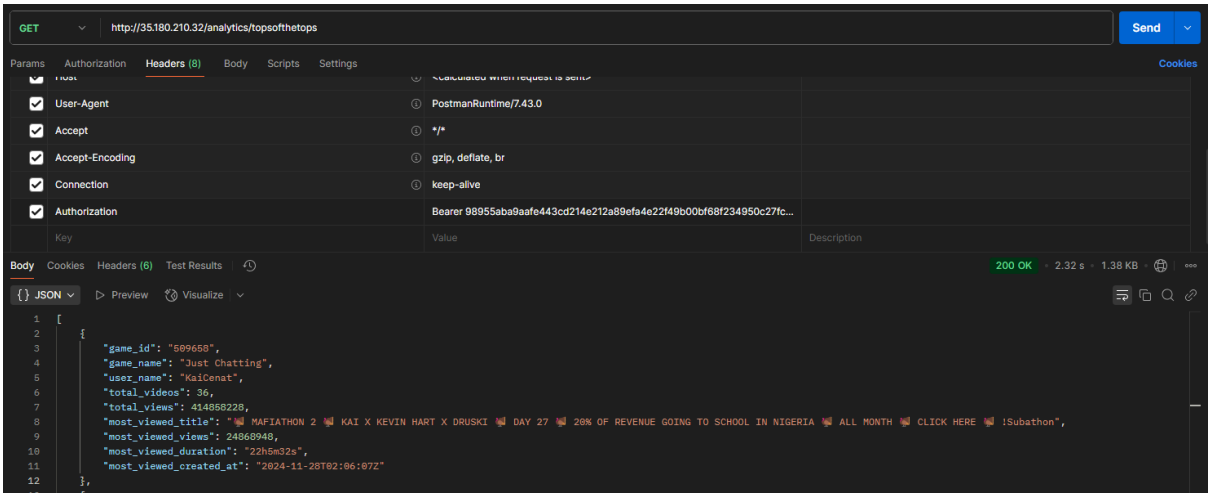
Ahora hemos generado nuestra api_key y podemos proceder a generar el token.

- **POST /token**: Dado un correo y su correspondiente api_key generamos un token de acceso a nuestra API:



Ahora ya tenemos nuestro token de acceso para realizar las peticiones GET.

- **GET analytics/user**: Solicitar la información de un usuario de twitch dado su ID.
  Para esta consulta usamos la siguiente URL: http://35.180.210.32/analytics/user?id=39518378
  En los headers tendremos que agregar nuestro token de acceso a la API, por ejemplo:
  Authorization Bearer token
  En esta url podemos modificar el id, para consultar la información de distintos usuarios.
- **GET analytics/streams**: Solicitar una lista de los directos que se están transmitiendo. Para ello, utilizaremos la siguiente url:
  http://35.180.210.32/analytics/streams
  También introduciremos nuestro token de acceso.
- **GET analytics/streams/enriched**: Solicitar una lista de los directos que se están transmitiendo ordenada de mayor a menor número de visualizaciones y con un límite de lista, es decir, si introducimos límite 3 se nos mostrarán los top 3 directos con más visualizaciones. Para ello utilizaremos la siguiente url: http://35.180.210.32/analytics/streams/enriched?limit=3
  En esta url podemos modificar el parámetro "limit" para seleccionar distintos tops; top 3, top 5, etc.
- **GET analytics/topsofthetops**: Solicitar información sobre los 40 videos más vistos de los top 3 juegos de twitch. Para esta petición podemos agregar el parámetro since, qué fuerza la actualización de los datos, es decir, si introducimos since=40, se comprueba si los datos que hay en la BBDD han sido actualizados hace menos de 40 segundos, y en caso de que no sea así, actualiza los datos.
  Para realizar esta petición hacemos uso de las siguientes urls:
  http://35.180.210.31/analytics/topsofthetops
  http://35.180.210.31/analytics/topsofthetops?since=40
  Como para todas las peticiones Get, debemos proporcionar nuestro token de acceso a la API en los headers.



Hemos lanzado la API haciendo uso de amazon lightsail.

Explicación del código

En el documento Entrega1.pdf habíamos explicado cómo funcionaban las siguientes peticiones:

- ➢ GET analytics/user
- ➢ GET analytics/streams
- ➢ GET analytics/streams/enriched

Ahora vamos a explicar las nuevas funcionalidades:

- ➢ POST /register
- ➢ POST /token
- ➢ GET analytics/topsofthetops

Además, vamos a explicar cómo hemos agregado la funcionalidad de autenticación con un token para realizar las peticiones GET y el uso de una nueva base de datos.

Para las dos peticiones POST, hemos generado una función `getBody()` en la cual obtenemos la información que se ha proporcionado en el body de la petición, es decir, el email y la api_key.

- ❖ POST /register: Hemos creado una función `registerUser()` la cual recibe el body de la petición decodificado. Lo que hacemos es obtener el email del body y generar una api_key. Después introducimos el email y su api_key a nuestra tabla *users* de la BBDD.
- ❖ POST /token: Hemos creado una función `generateToken()` que recibe el body de la petición decodificado. Primero obtenemos el email y la api_key del body y realizamos las comprobaciones necesarias. Después generamos un token y lo introducimos a nuestra tabla *tokens* de la BBDD. Además introducimos valores de created_at y expires_at, para manejar la expiración de los tokens.
- ❖ GET analytics/topsofthetops: Esta consulta es más compleja y la hemos dividido en las siguientes funciones:
  - **getTop3Games()**: Esta función simplemente realiza una llamada a la api de twitch para obtener los top 3 juegos.
  - **getTop40Videos()**: Esta función obtiene los top 40 videos de la categoría que se le proporciona a la función. Simplemente realizamos una llamada a la api de twitch: `https://api.twitch.tv/helix/videos?game_id=".$gameId."&sort=views&first=40.` Especificamos el game_id que hace referencia a la categoria la cual queremos obtener los videos; el parámetro sort que lo que hace es mostrar el resultado ordenado por visualizaciones; y el parámetro first que muestra solo los 40 primeros resultados.
  - **callTwitchAPI()**: Esta función obtiene los top 3 juegos llamando a la función `getTop3Games()`. Después para cada juego, llama a la función `getTop40Videos()` que obtiene los top 40 videos de cada juego. Para obtener los datos que nos interesan hemos tenido en cuenta el primer video, ya que, con la información del primer video obtenemos el user_name (usuario con el video más visto); most_viewed_title; most_viewed_views, etc. Para realizar el cálculo de total_videos y total_views hemos recorrido los 40 videos y hemos comprobado si esos videos pertenecian al usuario con el video más visto. En caso de ser así, sumamos lo necesario. Una vez hemos generado la respuesta con los datos que

queremos, llamamos a la función actualizarBDD() qué es la que se encarga de introducir estos datos en la tabla *top_games* de nuestra BBDD. Hemos dejado bastantes comentarios en el código para que se entienda.

- **getTopsofthetops()**: Esta función es la que se encarga de comprobar si es necesario llamar a la función callTwitchAPI o podemos mostrar los datos almacenados en la BBDD. Para ello hacemos una consulta a nuestra bbdd y miramos la fecha en la que se ha actualizado, después realizamos las comprobaciones necesarias dependiendo del parámetro since.

Para agregar estas funcionalidades hemos agregado sus casos en el index.php. Además, si la petición es de tipo GET, validamos que se haya introducido un token y que sea válido. Para ello, hacemos uso de la función validateToken() que se encuentra en el index.php.

Hemos creado también database.php que simplemente tiene dos funciones:

- **connectToDB()**: Se encarga de establecer la conexión con la bbdd.
- **actualizarBDD()**: Se encarga de introducir los datos de la petición topsofthetops a la tabla top_games de nuestra BBDD.

Estructura de la base de datos

USERS(id, email, api_key, created_at)
TWITCH_TOKENS(id, token, expires_at)
TOKENS(id, user_id, token, expires_at, created_at)
TOP_GAMES(id, game_id, game_name, user_name, total_videos, total_views, most_viewed_title, most_viewed_views, most_viewed_duration, most_viewed_created_at, updated_at)

Problemas y dificultades encontradas

Problemas en el código

1) Obtención de token de la API de twitch cada vez que se realiza una petición.
   En nuestro código anterior, cada vez que se realizaba una petición a nuestra API, generamos un token de acceso a la API de Twitch. Ahora, como tenemos una BBDD, hemos modificado la función getToken() para que solo se solicite un token si el que tenemos guardado en la base de datos ha expirado.

Problemas en el servidor

1) Problema de permisos: Mientras trabajabamos en local, la autenticación funcionaba correctamente. Sin embargo, al lanzarlo al servidor las peticiones devolvieron el siguiente error aún introduciendo un token correcto:
"Unauthorized: Token is invalid or expired"
El problema era que el servidor de Apache, por defecto, no pasa el header de Authorization al código PHP, por lo que no se podía obtener el token. Para solucionar esto, agregamos la siguiente línea en nuestro archivo .htaccess:

```
SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1
```

Lo que hace esta línea es capturar el header y guardarlo en la variable de entorno HTTP_AUTHORIZATION. De esta forma, PHP accede al header mediante $_SERVER['HTTP_AUTHORIZATION'].

Problemas en la BBDD

1) Error con los tipos de datos de las tablas: Al crear las tablas de la base de datos, los datos que debían ser int4 se crearon como uuid. El problema estuvo al intentar modificar el tipo de dato, y es que, hasta ese momento solo habíamos realizado consultas desde un Dataclip que más adelante nos dimos cuenta que solo se podía utilizar para hacer consultas de lectura. Para solucionar el problema al final hemos tenido que hacer uso de pgAdmin4 conectándonos a la base de datos y poder hacer consultas de lectura-escritura.

2) Registro de emails y obtención de API key: Si un correo ya registrado solicitaba una nueva API key, este se insertaba en la tabla sin comprobar previamente si existía. Aunque funcionaba la asignación de la API key, no tenía sentido ya que se almacenaba información redundante e inútil. Cuando nos dimos cuenta unos días más tarde, lo arreglamos mediante un UPDATE en vez de INSERT además de poner el atributo email como PK.

3) Formato del tiempo: Al realizar la consulta de /analytics/topofthetops, el parámetro since se introduce en segundos. Este parámetro debes ser comparado con el atributo updated_at de la base de datos. Tuvimos un conflicto entre tipos porque en la tabla el atributo se guarda en formato YYYY-MM-DD-HH-MM-S. Simplemente con unas pocas líneas de código para cambiar las variables de tipo lo arreglamos.

Implementación y ubicación del proyecto

Para probar las nuevas funcionalidades hay que utilizar alguna herramienta como POSTMAN. Podemos probar todas las peticiones de la siguiente forma (teniendo en cuenta que en cada petición de tipo GET debemos proporcionar un token):

- GET analytics/user: http://35.180.210.32/analytics/user?id=39518378

- GET analytics/streams: http://35.180.210.32/analytics/streams
- GET analytics/streams/enriched:http://35.180.210.32/analytics/streams/enriched?limit=3
- POST /register: http://35.180.210.32/register (introduciendo nuestro email en el body)
- POST /token: http://35.180.210.32/token (introduciendo nuestro email y api_key en el body)
- GET analytics/topsofthetops: http://35.180.210.32/analytics/topsofthetops

Para trabajar con nuestro proyecto, como se explica en el readme, simplemente hacemos un pull del repositorio de github a nuestros ordenadores y modificamos el código necesario. Después, lo probamos en local de la siguiente manera (ejemplo con la terminal de phpStorm):

```
PS C:\Users\Ainhoa\PhpstormProjects\twitch-analytics> php -S localhost:12000
[Fri Feb  7 15:21:53 2025] PHP 8.4.3 Development Server (http://localhost:12000) started
```

Después, simplemente realizamos las peticiones necesarias a esa url desde postman. Por ejemplo: http://localhost:12000/analytics/topsofthetops (Se necesita introducir el token en el header).

Cuando nos aseguramos de que todo funcione, para hacer que funcione el servidor público haremos lo siguiente:

1) Realizar un commit a nuestro repositorio de github con los cambios.
2) Acceder a nuestra instancia en la página web de amazon lightsail.
3) Realizar un pull de github con los cambios actualizados.

El servidor almacena todo el código de php en la carpeta htdocs. Esta carpeta la podemos encontrar navegando por la terminal de nuestro servidor a la siguiente ruta: cd htdocs